

Weight Sparsification for Efficient Neural Network Compression and Hardware-Aware Optimization

Muhammad Usman Nadeem
Utah State University
Email: usman.nadeem@usu.edu

Abstract—Modern neural networks are highly over-parameterized, often containing millions of weights, many of which contribute little to the final prediction. This paper studies *weight sparsification*—the removal of low-importance weights after training—from both an algorithmic and hardware perspective. Using PyTorch on NVIDIA GPUs, we compare unstructured and structured pruning on an image classification task (ResNet-18 on CIFAR-10) and unstructured pruning on a language modeling task (DistilGPT-2 on WikiText). We measure test accuracy / perplexity, model size, inference latency, and GPU memory usage. On CIFAR-10, magnitude-based unstructured pruning achieves high logical sparsity with only minor accuracy degradation, while structured channel pruning maintains accuracy up to 50% sparsity with a small drop at the highest pruning level. Both schemes provide modest latency improvements (up to about 5%) and large peak memory reductions (around 80%) relative to the dense baseline. On the language model, unstructured pruning maintains reasonable perplexity at moderate sparsity but can temporarily increase memory usage when masks are stored as separate parameters. Our results highlight that sparsity alone is not sufficient: the *pattern* of sparsity and the framework’s implementation determine whether speed and memory benefits are realized. We conclude by discussing semi-structured 2:4 sparsity on Ampere GPUs as a promising hardware-aware direction.

I. INTRODUCTION

Deep neural networks (DNNs) achieve state-of-the-art performance in vision, language, and speech, but their success is accompanied by large parameter counts and high computational demands. Storing and moving these parameters dominates memory footprint and energy consumption in many deployments, especially on resource-constrained or latency-critical systems. However, empirical and theoretical studies show that many weights are redundant: they can be removed or quantized with little impact on accuracy.

Weight sparsification (or pruning) addresses this redundancy by removing unimportant connections from a trained model and optionally re-training the remaining weights. This can reduce model size, memory traffic, and multiply-accumulate (MAC) operations. In practice, the benefit depends not only on how many weights are pruned but also on how sparsity is represented and executed on the underlying hardware.

This project investigates weight sparsification on NVIDIA GPUs from two angles:

- **Algorithmic:** How do different pruning schemes affect test accuracy for vision and language tasks?
- **Hardware-aware:** When does sparsity translate into real gains in GPU latency and memory usage?

We focus on unstructured vs. structured pruning for a CIFAR-10 ResNet-18 classifier, and on unstructured pruning of DistilGPT-2 for WikiText language modeling.

II. RELATED WORK

Early work by LeCun *et al.* introduced *Optimal Brain Damage*, which used second-order derivatives to prune weights with minimal impact on loss [2]. Han *et al.* later popularized magnitude-based pruning with iterative prune-and-re-train cycles, demonstrating large compression ratios for CNNs [1]. Frankle and Carbin proposed the Lottery Ticket Hypothesis, showing that sparse subnetworks inside dense models can train to full accuracy when reinitialized properly [3].

Most of this literature evaluates algorithmic performance (accuracy, sparsity, compression) but gives comparatively less detail on how sparsity interacts with GPU kernels, memory allocation, and hardware features. Unstructured sparsity, where individual weights are zeroed arbitrarily, creates irregular memory access and typically requires specialized sparse kernels to exploit. Structured sparsity (pruning whole channels or filters) preserves dense structure and is more compatible with standard libraries such as cuDNN.

More recently, NVIDIA introduced *semi-structured* 2:4 sparsity in the Ampere architecture: in every group of four weights, exactly two must be non-zero. Tensor Cores and the cuSPARSELt library can exploit this pattern to achieve up to $2\times$ higher throughput and reduced memory footprint [4]. Our work does not implement 2:4 sparsity directly but uses it as a motivation for hardware-aware pruning patterns.

III. PROPOSED METHOD

A. Sparsification Pipeline

Our sparsification pipeline follows a standard four-stage flow:

- 1) Train a dense baseline model to convergence.
- 2) Rank weights or channels by importance (e.g., L_1 magnitude).
- 3) Prune a specified fraction according to the ranking.
- 4) Re-train the pruned model for several epochs.

This pipeline is applied to both vision (ResNet-18 on CIFAR-10) and language (DistilGPT-2 on WikiText) tasks.

B. Unstructured Pruning

Unstructured pruning removes individual weights based on their importance, without enforcing any particular structure on the remaining non-zeros. In this work, we adopt a simple magnitude-based criterion: weights with small absolute value are considered less important and are preferentially removed. For CIFAR-10, we apply unstructured pruning to the convolutional and fully-connected layers of the ResNet-18 model; for WikiText, we prune the linear and embedding layers of DistilGPT-2. This produces irregular, fine-grained sparsity patterns where zeros are scattered throughout each tensor.

In PyTorch, pruning is applied using L_1 -norm based utilities that internally rely on *reparameterization*. Instead of immediately shrinking the layer, the framework keeps the original dense weights and introduces a binary mask that is multiplied with them at run time. From an algorithmic standpoint, this is convenient because the same layer interface is preserved and pruned weights can be re-trained. However, from a hardware perspective it has two important consequences: (i) memory usage reflects the storage of both the original weights and the mask rather than just the surviving weights, and (ii) the underlying kernels still operate on dense tensors, merely skipping contributions that have been multiplied by zero. As a result, unstructured pruning in this default form primarily changes the *logical* sparsity of the model, while its physical memory footprint and execution pattern on the GPU remain largely dense.

C. Structured Channel Pruning

Structured pruning enforces sparsity at the level of whole channels or filters, rather than individual weights. In convolutional networks such as ResNet-18 on CIFAR-10, this means removing complete output channels (and their corresponding kernels) from selected convolutional layers. We rank filters by the L_1 norm of their weights and prune a fixed fraction of the least important filters in each prunable layer. This procedure directly reduces the number of output feature maps and, transitively, the number of input channels seen by subsequent layers.

Crucially, after structured pruning we remove the reparameterization and materialize the pruned architecture as a smaller dense network: convolutional layers are stored and executed with fewer channels, and activation tensors shrink accordingly. Unlike unstructured sparsity, this yields changes that are visible to cuDNN and the underlying A100 GPU: kernel launch dimensions become smaller, memory allocations are reduced, and fewer MAC operations are issued. Structured pruning therefore produces sparsity that is naturally compatible with standard dense kernels, translating more directly into reduced memory traffic and potential latency improvements.

D. Hardware-Aware Perspective

Our method is not a new pruning algorithm but a systematic comparison of pruning *patterns* and their hardware implications:

- **Unstructured sparsity** offers flexibility and high logical compression but, without custom sparse kernels, usually provides only modest speed or memory gains.
- **Structured sparsity** aligns with dense GPU kernels by shrinking tensor shapes, which can more directly reduce memory traffic and computation.
- **Semi-structured 2:4 sparsity** (not implemented here) is an example of hardware co-design, where the sparsity pattern is constrained to what Tensor Cores can efficiently exploit [4].

IV. EXPERIMENTS

This section describes the datasets, baseline models and hardware platform, pruning configurations, evaluation metrics, and the resulting accuracy, perplexity, latency, and memory trends for both vision and language tasks.

A. Datasets

We evaluate on one vision and one language dataset:

CIFAR-10: 50 000 training and 10 000 test images of size 32×32 RGB across 10 classes. We use the standard train/test split and apply common data augmentation (random horizontal flips and random crops) during training.

WikiText: DistilGPT-2 is re-trained on the WikiText corpus (WikiText-2 or similar), a word-level dataset containing several million tokens from Wikipedia articles. Models are evaluated using perplexity on validation and test splits, as is standard for language modeling.

B. Baseline Models and Hardware

For CIFAR-10, the baseline model is a standard **ResNet-18** adapted to 32×32 inputs. The dense model has approximately **11.2 million parameters** and achieves **94.60%** test accuracy after training.

For WikiText, the baseline is **DistilGPT-2**, a distilled transformer-based language model with about **82 million parameters**. The dense model attains a test perplexity of **40.53** on the WikiText benchmark.

All experiments are implemented in PyTorch and executed on a single **NVIDIA A100 GPU** in **Google Colab**.¹ The A100 provides ample compute and memory bandwidth, so observed differences in performance are dominated by the pruning configuration rather than hardware limitations. CUDA and cuDNN are used for all dense convolution and matrix multiplication kernels.

C. Pruning Configurations and Metrics

For ResNet-18 on CIFAR-10, we consider two pruning schemes:

- **Unstructured pruning** of convolutional and fully-connected weights at sparsity levels of 30%, 50%, and 70%.

¹Source code for all experiments is available at: <https://github.com/usmannadeem31/ANN-Project>.

TABLE I
CIFAR-10 PRUNING SUMMARY: ACCURACY VS. SPARSITY (RESNET-18).

Method	Sparsity (%)	Acc Before (%)	Acc After (%)	Norm Acc (% of base)
Baseline (Dense)	0	94.60	94.60	1.00
Unstructured	30	94.29	94.91	1.00
Unstructured	50	91.84	94.34	1.00
Unstructured	70	54.29	94.98	1.00
Structured	20	12.45	94.36	1.00
Structured	30	10.00	94.35	1.00
Structured	50	10.00	92.12	0.97

- **Structured channel pruning** of convolutional filters at sparsity levels of 20%, 30%, and 50%, measured as the fraction of pruned output channels.

For DistilGPT-2 on WikiText, we apply:

- **Unstructured pruning** of linear and embedding weights at 21% and 33% sparsity.
- **Structured pruning** of transformer dimensions at 9–12% sparsity to study its impact on perplexity and stability.

Across both tasks, we report:

- **Algorithmic metrics:** test accuracy (CIFAR-10) and test perplexity (WikiText), before and after re-training.
- **Compression metrics:** number of trainable parameters and on-disk model size.
- **Hardware metrics:** inference latency and peak GPU memory usage.

Latency is reported in milliseconds per image (CIFAR-10) or milliseconds per token (WikiText). Peak GPU memory is measured using PyTorch memory counters and cross-checked with `nvidia-smi`. Speedup is defined as the ratio of baseline latency to pruned latency, and memory reduction is the percentage change in peak memory relative to the dense baseline (negative values indicate an increase).

D. CIFAR-10 Accuracy and Sparsity

Table I summarizes the effect of pruning on CIFAR-10 accuracy. We report sparsity, accuracy before re-training, accuracy after re-training, and normalized accuracy relative to the dense baseline.

Figure 1 visualizes test accuracy versus sparsity for both pruning schemes. Several trends are evident:

- Moderate sparsity (20–30%) is essentially “free” for both unstructured and structured pruning: accuracy remains within about ± 0.3 percentage points of the 94.60% baseline.
- Unstructured pruning at 70% sparsity achieves the highest post-pruning accuracy (94.98%), slightly exceeding the baseline, likely due to a mild regularization effect during re-training.
- Structured pruning remains competitive up to 30% sparsity, but at 50% channel pruning the test accuracy drops to 92.12%, corresponding to a normalized accuracy of 0.97.

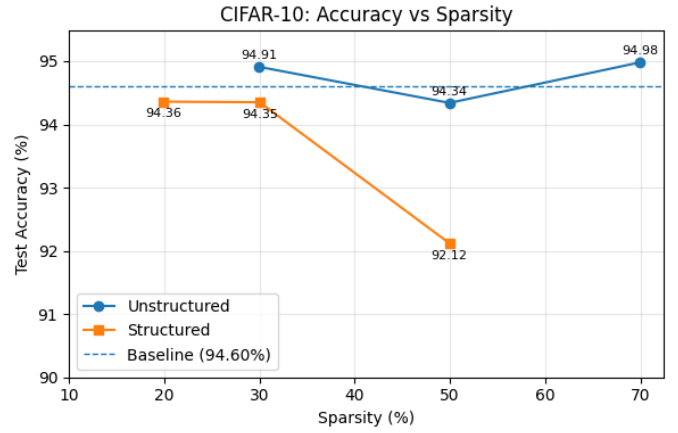


Fig. 1. CIFAR-10 (ResNet-18): accuracy vs. sparsity for unstructured and structured pruning. The dashed line indicates the dense baseline accuracy (94.60%).

TABLE II
CIFAR-10 HARDWARE METRICS: LATENCY AND PEAK GPU MEMORY (RESNET-18).

Method	Sparsity (%)	Latency (ms/img)	Peak Mem (MB)	Speedup ($\times$ vs base)	Mem Red. (%)
Baseline (Dense)	0	1.057	3568.7	1.000	0.0
Unstructured	30	1.016	707.0	1.041	80.2
Unstructured	50	1.030	707.6	1.027	80.2
Unstructured	70	1.027	707.0	1.030	80.2
Structured	20	1.005	796.5	1.052	77.7
Structured	30	1.029	796.7	1.028	77.7
Structured	50	1.033	795.7	1.024	77.7

E. CIFAR-10 Hardware Results

Table II reports inference latency and peak GPU memory for CIFAR-10, along with speedup and memory reduction relative to the dense baseline. Figures 2 and 3 show the same hardware trends as line plots.

Relative to the dense baseline, both unstructured and structured pruning provide modest speedups and large peak memory reductions:

- The best latency (1.005 ms/image) is obtained by structured pruning at 20% sparsity, corresponding to a 5.2% speedup over the 1.057 ms/image baseline. The best unstructured configuration (30% sparsity) achieves a similar 4.1% speedup.
- Peak GPU memory decreases from 3568.7 MB to roughly 700–800 MB for all pruned variants, an 80% reduction. The difference between unstructured and structured pruning in this regime is small; both benefit from smaller activations and fewer parameters during inference on the A100.
- Beyond 30% sparsity, further changes in latency and memory are marginal, indicating that for this ResNet-18 model the A100 remains largely memory-bound rather than compute-bound.

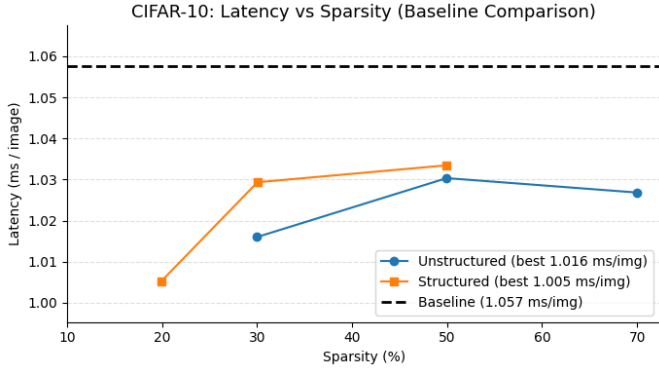


Fig. 2. CIFAR-10 (ResNet-18): inference latency vs. sparsity.

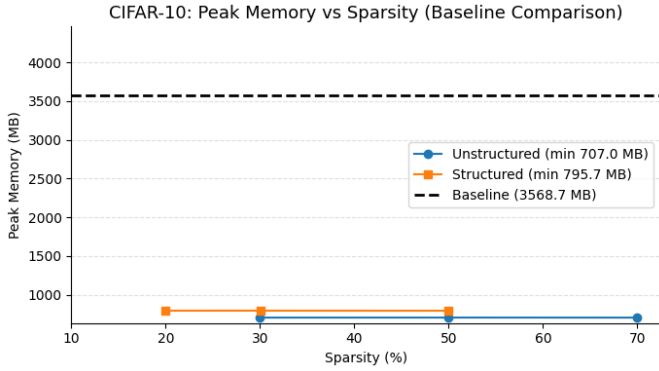


Fig. 3. CIFAR-10 (ResNet-18): peak GPU memory vs. sparsity.

F. WikiText Perplexity and Hardware Behavior

Table III summarizes the impact of pruning on DistilGPT-2 perplexity.

Unstructured pruning is relatively gentle on DistilGPT-2: at 21% sparsity, perplexity increases only slightly (from 40.53 to 41.66), and even at 33% sparsity the final perplexity of 43.60 remains within a usable range. In contrast, structured pruning severely damages the model: both 9% and 12% structured sparsity cause the pre-re-training perplexity to blow up to 10^9 -scale values, and even after re-training, the final perplexities (336.49 and 505.07) are far from the dense baseline. This indicates that pruning entire channels in transformer layers is much more disruptive than pruning individual weights.

Hardware behavior for WikiText is reported in Table IV.

Latency improves slightly for all pruned variants (up to about 4.6% speedup), but peak GPU memory *increases* by roughly 10%, which is reflected as negative values in the “Mem Red.” column. This behavior is a direct consequence of PyTorch’s reparameterization: each pruned layer stores both the original dense weights and an additional binary mask. Unless the masks are removed and the layer is materialized as a smaller dense tensor, the total parameter footprint grows despite the higher logical sparsity.

TABLE III
WIKITEXT LANGUAGE MODELING (DISTILGPT-2): PERPLEXITY VS. SPARSITY.

Method	Sparsity (%)	Perplexity Before	Perplexity After
Baseline (Dense)	0	40.53	40.53
Unstructured	21	41.73	41.66
Unstructured	33	50.95	43.60
Structured	12	1.85×10^9	505.07
Structured	9	1.35×10^9	336.49

TABLE IV
WIKITEXT HARDWARE METRICS (DISTILGPT-2): LATENCY AND PEAK GPU MEMORY. NEGATIVE VALUES IN THE MEMORY REDUCTION COLUMN INDICATE AN *increase* IN MEMORY USAGE (I.E., NO REDUCTION) CAUSED BY STORING PRUNING MASKS ALONGSIDE THE ORIGINAL WEIGHTS.

Method	Sparsity (%)	Latency (ms/token)	Peak Mem (MB)	Speedup ($\times$ vs base)	Mem Red. (%)
Baseline (Dense)	0	0.011433	3562.7	1.000	0.0
Unstructured	21	0.011314	3929.6	1.011	−10.3
Unstructured	33	0.010948	3929.4	1.044	−10.3
Structured	9	0.010946	3929.3	1.044	−10.3
Structured	12	0.010927	3928.7	1.046	−10.3

G. Discussion

Across both tasks, unstructured pruning is effective at achieving high logical sparsity with modest accuracy or perplexity degradation. Structured channel pruning works well for the ResNet-18 CIFAR-10 model up to 30% sparsity, providing similar accuracy and hardware metrics to unstructured pruning, but even mild structured pruning is highly destructive for DistilGPT-2 on WikiText.

The CIFAR-10 experiments show that simple pruning schemes can yield small but measurable speedups and large peak memory reductions when deployed on modern GPUs such as the A100. The WikiText experiments emphasize the importance of hardware- and framework-aware design: without removing pruning masks or using sparse kernels, sparsity can counterintuitively increase memory usage instead of reducing it.

V. CONCLUSION AND FUTURE WORK

This project presented an empirical study of weight sparsification for both vision and language models, with particular focus on how pruning patterns interact with GPU hardware. Magnitude-based unstructured pruning achieved high sparsity levels on CIFAR-10 and WikiText with modest accuracy and perplexity degradation. Structured channel pruning of ResNet-18 produced comparable accuracy up to 30% sparsity and delivered similar hardware gains. For DistilGPT-2, however, even small amounts of structured pruning severely harmed perplexity, underscoring the sensitivity of transformer architectures to channel-level pruning.

Our results support the view that sparsity must be *hardware-aware*: simply increasing the percentage of zeros is not enough. Patterns that align with dense kernels (structured pruning) or with architecture-specific features (such as NVIDIA’s

2:4 semi-structured sparsity on Ampere GPUs) are more promising for deployment. As future work, we plan to:

- Implement semi-structured 2:4 pruning and evaluate it on Ampere GPUs using cuSPARSELt.
- Explore layer-wise and sensitivity-aware pruning schedules for language models to avoid catastrophic perplexity degradation.
- Combine sparsity with other compression techniques such as quantization to further reduce memory and energy consumption.

REFERENCES

- [1] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural networks," in *Proc. NIPS*, 2015.
- [2] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage," in *Proc. NIPS*, 1989.
- [3] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. ICLR*, 2019.
- [4] NVIDIA, "Ampere architecture in-depth: Structured sparsity support," NVIDIA Technical Report, 2021.